

Drowsiness Detection Project Report

Dinh Duc Dat Nguyen
Oregon State University
datndd260897@gmail.com

Feng-Shih Cheng
Oregon State University
karaburi2023@gmail.com

Abstract

Driver fatigue is a leading cause of traffic accidents everywhere. This project develops a real-time drowsiness detection system using a lightweight convolutional neural network (CNN) for binary eye-state classification (open vs. closed). To ensure efficiency for resource-constrained environments, the system utilizes Haar Cascade classifiers for eye localization and a temporal scoring mechanism to distinguish normal blinks from prolonged eye closure. The model, trained on the Yawn-Eye and MRL Eye datasets, achieved a validation accuracy of 98.14%, with a sensitivity of 97.92% for closed-eye detection and a specificity of 98.36% for open-eye recognition. While experimental results show strong predictive performance, live testing revealed that the current localization method is sensitive to complex backgrounds and lighting changes. These findings suggest that while the lightweight CNN approach is effective for classification, practical deployment requires more robust tracking and broader testing across diverse driving conditions.

1. Introduction

As the pace of modern society continues to accelerate, driver fatigue caused by insufficient sleep has become one of the critical road-safety issues. In the United States, the AAA Foundation for Traffic Safety reported in 2016 that crash risk increases greatly as sleep duration decreases. For example, drivers who sleep 5 to 6 hours have 1.9 times higher crash risk relative to fully rested drivers, rising to 4.3 times higher for those who sleep only 4 to 5 hours [11].

In the United States, 17.6% of fatal crashes between 2017 and 2021 involved drowsy driving [12]. This issue remains ongoing, and a 2024 survey found that 25% of drivers admitted to having engaged in drowsy driving within the previous 30 days [13]. Additionally, in Colorado, the Colorado State Patrol reported 627 drowsy-driving-related incidents as of November 2025 [8].

Though car manufacturers such as Tesla and Mercedes-Benz have incorporated advanced driver-assistance systems

(ADAS) into their vehicles [1, 5], accurately assessing a driver’s real-time physiological and cognitive state through non-invasive methods remains a significant and unresolved challenge in the field of modern automotive safety.

In order to successfully prevent traffic accidents, detecting drowsiness is a fundamental and valuable requirement. Traditional drowsiness-detection methods often rely on handcrafted indicators such as the Percentage of Eye Closure (PERCLOS) [3]. These approaches typically combine manually designed features with threshold-based decision rules. However, their performance can degrade under complex lighting conditions, as eye appearance varies significantly from person to person. To address this issue, we develop a deep learning eye-state classification model that tracks eye-opening and eye-closing patterns to detect drowsiness in real time.

Inspired by Hassan et al. [4], this project adopts a simpler approach to develop a simplified deep learning eye-state classification model for binary classification (open vs. closed), trained on the Yawn-Eye Dataset [9] and the MRL Eye Dataset, and deploys the best-performing model for real-time eye-state monitoring. Eye regions are localized in real time using pre-trained OpenCV Haar Cascade classifiers [7], avoiding the computational overhead of facial landmark estimation. The proposed method consists of data preparation, image normalization and augmentation, CNN model design and training, and evaluation using standard classification metrics to identify the best-performing model. Experimental results show that the method reaches a validation accuracy of 98.14%, indicating strong predictive performance.

The rest of the paper is structured as follows: In “Background” section, we introduce the background knowledge needed for this study. In “Related Works” section, we review prior studies on driver drowsiness detection. In “Technical Approach” section, we describe the proposed methodology, including the network architecture and training strategy. In “Experiments” section, we present the experimental setup, evaluation metrics, and results. In “Conclusion and Future Work” section, we summarize the findings and discuss possible directions for future improvement.

2. Background

This study focuses on binary eye-state classification using computer vision, where each eye image is assigned one of two classes: *open* or *closed*. In practical drowsiness monitoring, prolonged eye closure is a strong behavioral indicator of fatigue and is commonly quantified by metrics such as PERCLOS [3]. Instead of manually defining eye-related shape features, the model learns useful visual patterns directly from raw images.

Convolutional Neural Networks (CNNs) are particularly suitable for this task because local convolution kernels capture eye features at different spatial scales. Pooling operations improve tolerance to small shifts, while Batch Normalization and ReLU help stable and efficient optimization. For real-time use on limited hardware, lightweight CNNs are usually preferred because they balance accuracy and computing resource.

To achieve real-time localization before classification, Haar Cascade classifiers [7], provide a computationally efficient solution. These classifiers use simple Haar-like features and a cascade of boosted decision trees to rapidly filter out non-eye regions, enabling low-latency operation on live video streams.

Formally, given a preprocessed eye image $x \in \mathbb{R}^{1 \times 24 \times 24}$ localized by the cascade classifier, the model predicts class probabilities

$$\hat{p}(y | x) = f_{\theta}(x), \quad y \in \{0, 1\}. \quad (1)$$

where $y = 0$ denotes *closed eye* and $y = 1$ denotes *open eye*. The training objective is to find parameters θ that minimize cross-entropy loss over the dataset:

$$\min_{\theta} \frac{1}{N} \sum_{n=1}^N \mathcal{L}_{CE}(y_n, f_{\theta}(x_n)). \quad (2)$$

where N is the number of training samples, y_n is the ground-truth label of sample n , and $\mathcal{L}_{CE}$ denotes the cross-entropy loss.

In practice, a single frame can be misclassified because of blinking, the eye being partly blocked, or changes in lighting. To reduce these errors, the system uses a temporal score that accumulates consecutive closed-eye detections. This helps filter out normal blinks and triggers an alarm only when the eyes stay closed for a longer period.

3. Related Works

Several studies have developed real-time eye state recognition systems designed for resource-constrained embedded devices.

For example, DriCare represents a great advancement in fatigue detection using video-based facial analysis [2]. To overcome the tracking inaccuracies that previous algorithms

suffered from, the authors introduced a new method, which enables more reliable face tracking. The core of DriCare’s improvement lies in detecting 68 major facial characteristics to define facial regions. By analyzing temporal cues like yawning frequency, blinking rate and the duration of eye closure, the system can effectively identify signs of fatigue. Experimental results demonstrate that DriCare achieves a high accuracy of approximately 92%.

To mitigate limitations of visible-light cameras under fluctuating light and vision, a study has introduced facial thermal imaging and infrared-based illuminators [6]. These approaches evaluate drowsiness by extracting metrics such as Percentage of Eye Closure (PERCLOS), Eye Closing Duration (ECD) and Average Mouth Opening Time (AOT), along with physiological features like inspiratory-to-Expiratory ratio. By using a fuzzy logic framework and SVM classifiers, the approach categorizes fatigue into three levels with up to 91% accuracy.

More recently, deep learning has made a lot progress in eye-state classification. Transformer-based architectures, for instance, have pushed accuracy extremely high, reaching 99.15% on the MRL Eye Dataset using a Vision Transformer (ViT) [4]. Other researchers have also successfully used heavy convolutional networks like ResNet50 and Xception [1]. However, deploying these massive models in real-world automotive environments might be challenging because they require more memory and computational power, which increases deployment cost on cameras and mobile devices. Several studies have thus focused on real-time eye state recognition systems specifically designed for resource-constrained embedded devices such as Android [5]. Inspired by this line of work, our project uses a lighter technical stack instead of heavy Transformer models. We follow this direction to keep the system practical for resource-constrained in vehicle environment.

4. Technical Approach

In this project, we implement a lightweight CNN specifically optimized for binary eye-state classification. The architecture consists of three convolutional layers designed to extract spatial features from eye images effectively.

4.1. Neural Network Architecture

The proposed model is designed from scratch to prioritize low-latency, real-time inference. The network accepts input images with a resolution of $1 \times 24 \times 24$ pixels and processes them through three sequential convolutional layers with 32, 64, and 128 filters, respectively. These layers are specifically designed to extract high-level spatial features, such as the curvature of the eyelid and the geometric shape of the pupil. To ensure stable learning and faster convergence, each convolutional layer is followed by a Batch Normalization layer. Nonlinearity is introduced through the Recti-

fied Linear Unit (ReLU) activation function, which enables the model to capture complex visual patterns. Additionally, Max Pooling layers are applied after each convolution to reduce the spatial dimensions of the feature maps, focusing the network on the most important visual information.

Following the feature extraction stage, the resulting multi-dimensional data is flattened into a feature vector consisting of 1152 units. This vector is then passed into a fully connected layer with 128 neurons. To mitigate the risk of overfitting and improve the model’s generalization capabilities, a dropout layer with a rate of 50% is implemented, which randomly deactivates half of the neurons during the training phase. The final output layer consists of two neurons, representing the "Closed" (0) and "Open" (1) eye states, respectively. This architecture ensures a quick and efficient classification process suitable for embedded systems.

4.2. Loss Function and Optimizer

Cross-Entropy was chosen instead of other functions like Mean Squared Error (MSE) because it is better for binary classification tasks. Unlike MSE, which can lead to vanishing gradients when predicted values are far from the target, Cross-Entropy contains a logarithmic scale that provides a significantly larger penalty for "confident but incorrect" predictions. This ensures better weight updates and faster convergence during the early stages of training.

$$L = - \sum_{i=1}^{C=2} y_i \log(\hat{y}_i) \quad (3)$$

where C denotes the total number of classes (which is 2 in this binary task), y_i is the ground truth indicator (0 or 1) for class i , and $\hat{y}_i$ is the predicted probability for class i .

The Adam optimizer is utilized with an initial learning rate of 0.001 for its adaptive learning rate capabilities. Furthermore, to ensure the model converges effectively and avoids local minima, a learning-rate scheduler is implemented. This scheduler monitors the validation loss and reduces the learning rate by a factor of 0.5 if no improvement is observed for two consecutive epochs, allowing for improved weight adjustments in the later stages of training.

5. Experiments

This section describes the experimental process, including the training pipeline and the real-time inference setup. We also explain the metrics used to evaluate the model and the specific hyperparameters chosen for training. Finally, we present the classification results on the validation set and discuss the findings from live tests.

5.1. Training Phase

The model is trained in Python using the PyTorch framework, and training was conducted on Google Colab with CUDA for hardware acceleration. The images used for training and validation are collected from two public datasets: the Yawn-Eye Dataset [9] and the MRL Eye Dataset [10]. The combined dataset is partitioned using an 80%-20% train-validation split, resulting in 13,759 images used for training and weight optimization, while the remaining samples serve as the validation set to monitor generalization performance.

To better handle varying environmental conditions, all eye images were converted to single-channel grayscale and resized to a fixed resolution of 24×24 pixels. During training, augmentation was applied by randomly adjusting brightness, contrast, saturation, and hue, along with random horizontal flipping and rotations of up to 15 degrees to account for natural head tilting. Finally, pixel values were normalized to make training more stable. Examples of the resulting preprocessed and augmented eye images are shown in Figure 1.

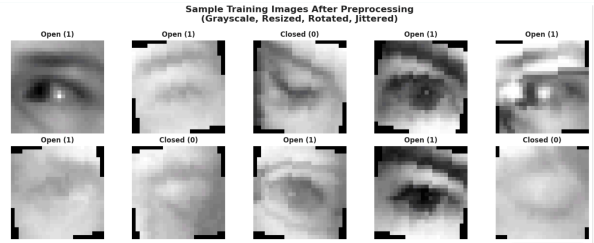


Figure 1. Examples of eye images after preprocessing and augmentation in the training pipeline.

5.2. Inference Phase

The real-time inference pipeline was tested locally on a Mac and operates directly on a live webcam stream. Eye regions are localized using pre-trained OpenCV Haar Cascade classifiers [7] instead of facial landmark detection. Each detected eye region is cropped, converted to grayscale, resized to 24×24 pixels, and rescaled before being passed to the trained model for inference. Figure 2 illustrates the system in operation, showing the localized eye regions and the accumulated drowsiness score in the lower-left corner.

To improve reliability in streaming conditions, the system uses a score-based filtering mechanism: it maintains a running score that increases when the driver’s eyes are closed and decreases when they are open. When the score hits 15, the system sounds an alarm to alert the driver.

5.3. Performance Evaluation Methods

In order to have a fair assessment, model performance is evaluated on a balanced validation set consisting of 1,732

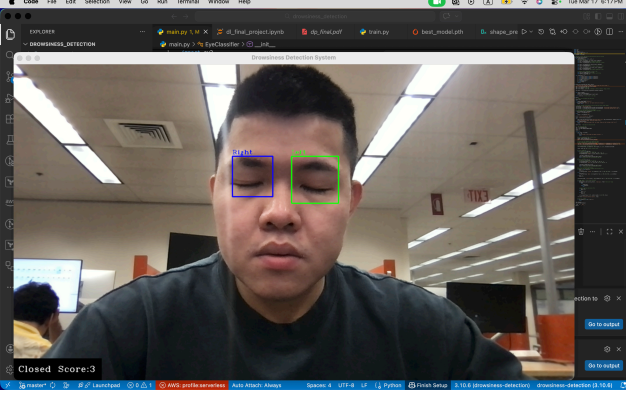


Figure 2. Real-time drowsiness detection interface showing localized eye regions and the accumulated temporal score.

closed-eye and 1,708 open-eye samples. This balanced distribution ensures that accuracy provides a reliable and unbiased measure of the model’s prediction. The evaluation metrics, accuracy, precision, sensitivity, and specificity, are defined as follows:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (4)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (5)$$

$$\text{Sensitivity} = \frac{TP}{TP + FN} \quad (6)$$

$$\text{Specificity} = \frac{TN}{TN + FP} \quad (7)$$

Furthermore, the system is qualitatively assessed through real-time testing to verify the effectiveness of the score-based filtering and alarm mechanism under live conditions.

5.4. Hyperparameters

The selection of hyperparameters was based on empirical testing to achieve reliable convergence. A batch size of 64 was utilized for training. The Adam optimizer was used with a fixed learning rate of 0.001 to enable rapid and self-adaptive convergence. The model was trained for a total of 20 epochs, using a dynamic learning rate reduction strategy to adjust the step size if the validation loss stagnated. Furthermore, a Cross-Entropy Loss function was implemented to minimize the classification error.

5.5. Experimental Results

After 20 epochs of training, the model achieved strong predictive performance, reaching a validation accuracy of 98.14%. This result indicates that the model successfully learned the structural differences between open and closed

eye states while maintaining good generalization on unseen data.

As shown in the following analysis, the model achieved a precision of 98.38% and a Sensitivity of 97.92% for the “Closed” state, along with a Specificity of 98.36% for the “Open” state. These metrics indicate that the architecture is quite reliable at detecting drowsiness events while maintaining high accuracy in identifying normal eye states.

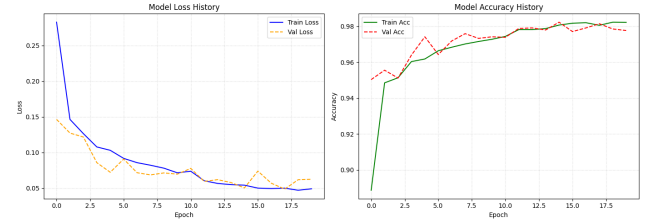


Figure 3. Training and validation loss and accuracy history over 20 epochs.

Figure 3 shows that both training and validation loss decrease rapidly in early epochs and then stabilize at low values, while both accuracy curves increase and converge near 99%. The model seem to generalize well on unseen samples and does not have clear overfitting under the current setting.

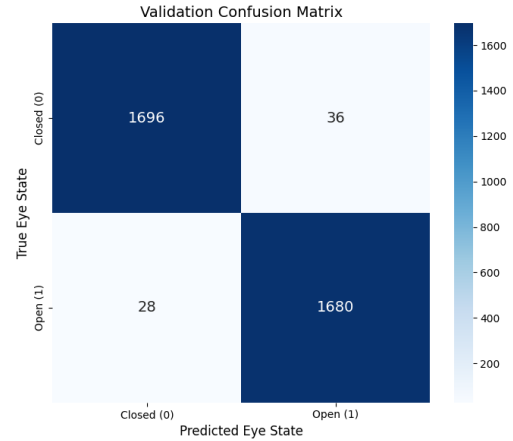


Figure 4. Confusion matrix of the model on the validation set.

Figure 4 is strongly concentrated on the diagonal entries, meaning that most open-eye and closed-eye samples are correctly classified; the very limited off-diagonal values indicate few false positives and false negatives.

To further evaluate the model’s behavior in safety-critical scenarios, we focus on three key metrics. Of all actual eye-closure events, the system successfully detects 97.92% (Sensitivity); of all actual open-eye moments, the system correctly identifies 98.36% (Specificity); and when the system triggers an eye-closure detection, it is accurate in 98.38% of cases (Precision). Figure 5 visualizes these metrics along with the Balanced Accuracy.

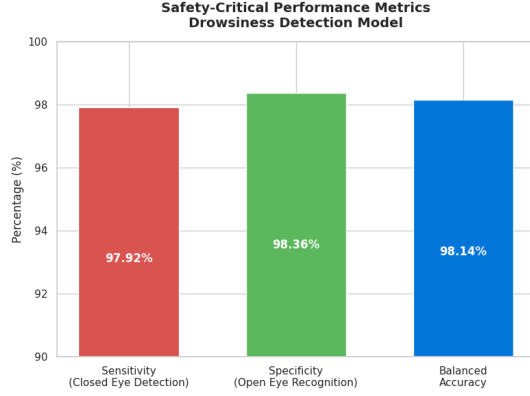


Figure 5. Safety-critical performance metrics: Sensitivity, Specificity, and Balanced Accuracy.

As summarized in Table 1, these results demonstrate that the model can be capable in both high detection rates for drowsiness and low false-alarm rates during normal driving.

Metric	Value
Precision (Closed Eye Detection)	98.38%
Sensitivity (Closed Eye Detection)	97.92%
Specificity (Open Eye Recognition)	98.36%
Balanced Accuracy	98.14%

Table 1. Safety-critical evaluation metrics on the validation set.

During live testing, the 15-frame smoothing threshold successfully separated normal blinks from prolonged eye closure. However, the system is not yet ready for actual road conditions. The Haar Cascade-based tracking often loses the target when the background is busy or when other people are in the frame. We also noted that the overall speed and stability under different lighting need more work. These tests show that while the prototype confirms the basic approach, it will need more stable tracking and more extensive testing before it can be used in real-world driving.

6. Conclusion and Future Work

This project built a lightweight CNN-based pipeline for drowsiness detection. The system accurately classifies eye states and uses temporal smoothing to reduce false alarms. While the results are encouraging, with a 98.14% validation accuracy, there are clear gaps in the current implementation. The eye-tracking method is sensitive to complex backgrounds and extra faces in the frame. Future work will involve testing the system in actual vehicles and replacing the simple detectors with more reliable alternatives that can handle low light and various head positions. Another key area is to expand the system to include yawn detection, as it provides another important indicator of fatigue. Finally, we plan to include more diverse training data, specifically eyes with glasses, as the current model's performance with

glasses is still unknown.

References

- [1] S. Abd El-Nabi, E.-S. M. El-Rabaie, K. F. Ramadan, and A. Emam. An efficient approach of drowsiness detection based on resnet50 and xception architectures. In *2023 3rd International Conference on Electronic Engineering (ICEEM)*, pages 1–6. IEEE, 2023.
- [2] W. Deng and R. Wu. Real-time driver-drowsiness detection system using facial features. *IEEE Access*, 7:118727–118738, 2019.
- [3] D. F. Dinges and R. Grace. PERCLOS: A valid psychophysiological measure of alertness as assessed by psychomotor vigilance. Technical Report FHWA-MCRT-98-006, United States. Federal Motor Carrier Safety Administration. Technology Division, Oct. 1998.
- [4] O. F. Hassan, A. F. Ibrahim, and B. Hafiz. Real-time driver drowsiness detection using transformer architectures: a novel deep learning approach. *Scientific Reports*, 15(17493), May 2025.
- [5] R. Jabbar, M. Shinoy, M. Kharbeche, K. Al-Khalifa, M. Krichen, and K. Barkaoui. Driver drowsiness detection model using convolutional neural networks techniques for android application, 2020.
- [6] S. E. H. Kiashari, A. Nahvi, H. Bakhoda, A. Homayounfard, and M. Tashakori. Evaluation of driver drowsiness using respiration analysis by thermal imaging on a driving simulator. *Multimedia Tools and Applications*, 79(25):17793–17815, 2020.
- [7] OpenCV Team. Opencv haar cascades data. <https://github.com/opencv/opencv/tree/2dd3d1371a9975248b1c236f4a94295a7ac5777d/data/haarcascades>, 2026. Accessed: 2026-03-16.
- [8] R. Patla. Over 600 drowsy driving crashes were reported in 2025; the state patrol shares the warning signs. <https://www.kkcollnews.com/2025/11/03/over-600-drowsy-driving-crashes-were-reported-2025-state-patrol-shares-warning-signs/>, November 3 2025. Accessed: 2026-03-04.
- [9] S. Raju. yawn_eye_dataset_new. <https://www.kaggle.com/datasets/serenaraju/yawn-eye-dataset-new>, 2020. Accessed: March 12, 2026.
- [10] A. Shingha. Mrl eye dataset. <https://www.kaggle.com/datasets/akashshingha850/mrl-eye-dataset>, 2023. Accessed: March 19, 2026.
- [11] B. C. Tefft. Acute sleep deprivation and risk of motor vehicle crash involvement. Technical report, AAA Foundation for Traffic Safety, Washington, D.C., Dec. 2016.
- [12] B. C. Tefft. Drowsy driving in fatal crashes, united states, 2017–2021. Research brief, AAA Foundation for Traffic Safety, Washington, D.C., March 2024.
- [13] X. Zhang and R. Steinbach. 2024 traffic safety culture index. Technical report, AAA Foundation for Traffic Safety, Washington, D.C., 2025.